Shortening Lean proofs in ProveIt

A source-grounded refactoring study of DiophantineConsistency

Prepared for Vladimir Reshetnikov · 26 October 2026

Executive assessment

The best opportunities are to share the algebraic and semantic interfaces that many existing proofs already implement. Replacing a substantial proof family with one invariant is more valuable than shortening individual tactic scripts. This development already contains good examples of that design: generic DAG node semantics, an addition-chain certificate checker, semiring gate semantics, and a structural proof of the actual tile permutation. Several other modules still duplicate the interfaces around those results.

This study combines an inventory of the active Lean tree, close reading of selected proof families, compiled refactoring prototypes, and reviewable patches. The deliverable is deliberately explicit about the difference between a complete patched module that has been checked, a checked generic helper whose downstream migration is still proposed, and an architectural idea that has not been implemented. It does not establish a new consistency equivalence or complete the remaining current-equation formalization.

Two complete checked patches are supplied: the package-52 coefficient-bound module decreases from 679 to 127 physical lines, and the representation module from 260 to 245. The latter saves 23 nonblank declaration lines after charging for its shared helpers. Six additional checked modules demonstrate the reusable interfaces and exact-signature alternatives discussed below.

The principal recommendations are:

1. Replace the repeated coefficient-envelope proof trees by one finite row split and a fixed arithmetic proof schema, preserving the sharp envelope.
2. Share the natural-assignment renaming law in representation constructors. The checked alternatives preserve the exact expressions and witness counts.
3. Use a scaled quotient/remainder identity for half-radix rotation, and a common residue API for cyclic block positions.
4. Share scanner iteration invariants and weighted finite-fiber summation.
5. Reuse the existing DAG and power-certificate interfaces to reduce generated proof scaffolding, measuring elaboration before removing cached declarations.
6. Reorganize expression and gate-semantics modules so each structural induction has one owner.

The two generated circuit-step files contain 6,567 physical lines. This is a large source-maintenance opportunity, but those lines are generated expansions, not 6,567 lines of distinct mathematical argument. No whole-project percentage reduction or compilation speedup is claimed.

Contents

1 Snapshot, scope, and measurement **3**
1.1 What counts as a useful simplification? . . . 3

2 Checked refactor: assignment naturality **3**
2.1 The repeated argument . . . 3
2.2 Measured result and compatibility . . . 4

3 Coefficient envelopes: one proof schema for all rows **4**
3.1 Why this is a substantial local target . . . 4
3.2 The proposed replacement structure . . . 5
3.3 Maintenance considerations . . . 6

4 Scaled division and cyclic residue interfaces **6**
4.1 An exact identity removes finite digit enumeration . . . 6
4.2 Factor the correct cyclic residue statements . . . 7

5 Shared scanner machinery without changing raw-input soundness **7**

6 One finite-fiber identity for folded stencils **8**

7 Generated circuits: reduce proof scaffolding with existing checkers **8**
7.1 Replace repetitive node interfaces selectively . . . 8
7.2 The power-label checker is already available . . . 9

8 Expression and compiler architecture **9**
8.1 Centralize residual-square semantics and degree bounds . . . 9
8.2 Move basic syntax facts below the consumers . . . 9
8.3 Share the gate soundness and completeness inductions . . . 10
8.4 Respect the object-language boundary . . . 10

9 Further targeted opportunities **10**
9.1 Bundle carry witnesses with named fields . . . 10
9.2 Use standard membership lemmas for small inductions . . . 11
9.3 Use existing structural proofs as patterns . . . 11

10 Implementation order and validation policy **11**
10.1 Trusted computation and simplification boundaries . . . 12

11 Validation results and reproducibility **12**
11.1 How to use the bundle . . . 13

12 Further research questions **13**

1 Snapshot, scope, and measurement

All repository observations refer to commit `60e9c5d620c6c38...`. The commit was retrieved from the public repository on 26 October 2026. Its message is "Prove exact cell translation, coarse field offsets and cycle exclusions." The project pins Lean `v4.32.0` and mathlib commit `81a5d257c8e410db227a6665ed08f64fea08e997`; the experiments use those exact versions. The upstream checkout was kept unchanged.

The requested topic also contains archived research and incoming material. The inventory in this report concerns the active `Computability/DiophantineConsistency/Lean` tree. Its main module directory includes some checks and audits as well as mathematical development; it should not be described as exclusively production proofs.

Measure	Active Lean tree	Main module directory
Lean files	546	363
Physical lines	73,971	68,308
Nonblank lines after comment removal	59,832	55,127
Lexically identified theorem/lemma declarations	4,967	4,902
Explicit `import Mathlib.Tactic` lines	8	8
`maxHeartbeats` option commands	134	134

These are reproducible lexical measurements, not counts of independent mathematical results or kernel proof-term sizes. The script removes nested Lean comments and preserves line boundaries. Declaration spans are estimates, because the script is not a Lean parser. Source hashes and per-file data are in `inventory/files.csv`; the script is `scripts/audit_inventory.py`.

The lexical scan found no `sorry`/`admit` tokens, explicit `axiom` commands, or `native_decide` tokens in the active tree. This is narrower than a transitive axiom audit. The supplied new proofs have their own actual `#print axioms` outputs; the entire existing development was not rebuilt or transitively audited for this report.

1.1 What counts as a useful simplification?

Four costs should be tracked separately: maintained source, mathematical proof structure, elaboration cost, and the transparency of the resulting theorem. The practical objective is a smaller set of reusable invariants with explicit interfaces. A one-line tactic that unfolds an enormous term can have worse elaboration behavior and be harder to repair than a longer sequence of named lemmas.

Exact expression syntax and witness counts matter here. A merely equivalent Diophantine representation can change the constants, size bounds, executable recipe, or later calibration. The representation refactor therefore includes equalities between the alternative constructors and the original constructors, proved by reflexivity. The coefficient refactor retains the literal coefficient function and the original envelope. The arithmetic interfaces keep all essential positivity and cyclic-domain assumptions.

2 Checked refactor: assignment naturality

2.1 The repeated argument

`Representation` repeatedly establishes that renaming input and witness indices commutes with the

natural-to-integer assignment. The same calculation appears in `Rep.reindex`, twice in `Rep.conjunction`, and twice in `Rep.disjunction`. The underlying law is

$$\mathrm{natAssignment}(v, w) \circ \mathrm{Sum.map}(f, g) = \mathrm{natAssignment}(v \circ f, w \circ g).$$

Its proof is function extensionality followed by the two constructors of a sum. Composing it with the existing `Expr.eval_rename` produces the semantic interface used at every call site.

```
theorem natAssignment_comp_sumMap (v : tau -> Nat) (w : beta -> Nat)
    (f : sigma -> tau) (g : alpha -> beta) :
    Function.comp (natAssignment v w) (Sum.map f g) =
      natAssignment (Function.comp v f) (Function.comp w g) := by
  funext i
  cases i <;> rfl
```

The complete, compilable source is `Lean/ProofSimplification/RepresentationNaturality.lean`.

The reindex correctness proof becomes an explicit simplification using the shared evaluation lemma. Conjunction and disjunction retain their existing `Fin.addCases` witness assembly. Their mathematical constructions are unchanged; only the repeated assignment calculation is centralized.

2.2 Measured result and compatibility

Declaration	Original nonblank lines	Refactored
`reindex`	12	5
`conjunction`	29	16
`disjunction`	30	17
Shared helper declarations	0	10
Total	71	48

The net saving in those declarations is 23 nonblank lines, or 32.4%. Including comments, spacing, and helper namespace scaffolding, the complete patched module has 245 physical lines instead of 260. Both measurements are reported so that omitted overhead is not mistaken for a saving.

Checked: the additive prototype, the original complete module, and the complete patched module compile. The alternative `reindex`, `conjunction`, and `disjunction` constructors equal the original constructors by `rfl`. Their stored syntax and witness count are definitionally identical; proof irrelevance identifies the correctness fields. The patch preserves public names. `git apply -check` succeeds against the pinned checkout.

The shared lemma is invoked explicitly. It is not necessary to make large expression definitions global simplification rules. The patch and the complete patched file are included separately, so reviewers can inspect either the change or the final module.

3 Coefficient envelopes: one proof schema for all rows

3.1 Why this is a substantial local target

`NativeCoefficientBounds` has 679 physical lines. Its documented subject is the package-52 native parent. Thus the largest measured local reduction belongs to that existing arithmetic layer; the current

length-2011 calibration has separate field and mask proofs. A large part of the file is the family `rowEnvelope_0` through `rowEnvelope_21`, followed by a public wrapper. Each private proof repeats three positivity facts and follows the conditional coefficient recipe through a tree of absolute-value bounds. The repeated region contains 158 occurrences of the same restricted `linarith only` closure and 136 applications of `abs_ite_le`.

Write the four ports as b, t, m, f, where t is the large transport parameter. The exact envelope is

$$E = 2b^2 + 2fb + m + f + 2b + tb + t + 8.$$

Given $b, t, m, f \geq 0$, the proof uses only those four inequalities and $b^2 \geq 0$, $fb \geq 0$, $tb \geq 0$. Once a coefficient branch is exposed, both sides of its absolute-value inequality are linear combinations of these quantities. Restricted linear arithmetic is therefore sufficient.

3.2 The proposed replacement structure

Keep the original definition of `envelope` and the actual `rowCoefficient`. Introduce the three shared nonnegativity facts once. Split the 22-valued row index, reduce that row's definition, and traverse its conditional tree with the existing semantic helper `abs_ite_le`. One `repeat' apply abs_ite_le` replaces the manually written branch tree. At every leaf, rewrite `abs_le` and solve the two inequalities with the same fixed collection of facts.

This removes proof duplication while retaining the sharp dependence on the transport port. In particular, the refactor does not replace the envelope by a convenient but larger quadratic expression in t. The downstream `envelope_lt` estimate and correction bound keep their mathematical content.

Checked: the complete patched module compiles and shrinks from 679 to 127 physical lines, a reduction of 552 lines (81.3%). All nine public declaration headers are unchanged. Both the shared envelope theorem and the published weighted bound have axiom reports containing only `propext`, `Classical.choice`, and `Quot.sound`. The combined theorem uses a locally scoped `maxHeartbeats 2000000` option: gathering 22 former proofs into one theorem changes resource budgeting. This is a maintained-source reduction, not a compilation-speed claim.

The shared proof body is:

```
have hsq := sq_nonneg (ports 0)
have hmf := mul_nonneg h3 h0
have hkp := mul_nonneg h1 h0
fin_cases k <;>
  simp only [rowCoefficient, envelope,
    Matrix.cons_val_zero', Matrix.cons_val_succ'] <;>
  repeat' apply abs_ite_le
all_goals
  rw [abs_le]
  constructor <;>
    linarith only [h0, h1, h2, h3, hsq, hmf, hkp]
```

The supplied patch has a complete module copy, not just an isolated proof fragment. Its public theorem has the same quantifiers, hypotheses, and conclusion. The code-generating origin of coefficient data does not make the arithmetic schema an oracle: branch reduction and `linarith` produce terms checked by Lean's kernel.

3.3 Maintenance considerations

The choice of branch interface matters: a trial using generic `split_ifs` reached a simplification-step limit on the longest row. Applying the shared absolute-value lemma handles both branch values without evaluating the pair-index equality tests or carrying their branch hypotheses. This is a concrete example where a small semantic lemma makes the concise proof more robust than unrestricted case splitting.

This proof depends deliberately on the finite recipe representation. If a row changes, Lean rechecks its branches against the same envelope and fails if the old bound no longer holds. Use `linarith only` with the fixed sign facts, so branch equalities do not become accidental proof dependencies. A named local tactic can be helpful if several coefficient families use the identical schema, but adding a general tactic framework for this one family is unnecessary.

A further, unimplemented improvement is to represent the repeated coefficient values by a small finite catalog. The baseline's 136 nondefault branches have 25 distinct textual value expressions, plus the default zero. The row/pair lookup would return a catalog entry, and its envelope bound would be proved once per entry. A separate exact evaluation theorem must identify the catalog lookup with the existing literal coefficient function. That extra equality is what makes the catalog a proof refactor rather than a changed arithmetic encoding. This is a larger generator and data-layout change than the immediate proof-schema patch.

4 Scaled division and cyclic residue interfaces

4.1 An exact identity removes finite digit enumeration

For natural numbers s, x, y with $s > 0$,

$$(sx) \bmod (2s) + \left\lfloor \frac{sy}{2s} \right\rfloor = s(x \bmod 2) + \left\lfloor \frac{y}{2} \right\rfloor . \tag{1}$$

This follows directly from `Nat.mul_mod_mul_left` and `Nat.mul_div_mul_left`. It is an equality for arbitrary x, y; no digit bound is required. The assumption $s > 0$ is essential: at $s = 0$ the left side is zero, whereas the right side need not be.

If $y \leq 3$, then $x \bmod 2 \leq 1$ and $y/2 \leq 1$, giving

$$(sx) \bmod (2s) + (sy)/(2s) \leq s + 1.$$

If s is even as well, the parity of the same expression is exactly $y/2$. These are `scaled_half_bound` and `scaled_half_parity` in the supplied `ArithmeticRefactors` module. Notice that the bound on x is unnecessary; only the predecessor value y needs its small-digit bound.

`NativeCyclicRotation` currently proves `shiftDigits_half_bound` using four cases for the current digit and a separate quotient estimate. Its `shiftDigits_half_parity_89` uses the sixteen combinations of two four-valued digits. Equation (1) explains both arguments at once. The supplied `RotationRefactors` alternatives use the same original theorem signatures and retain the absence of a `NeZero n` assumption. They do not call the old bound or parity theorem.

Checked: both exact-signature alternatives compile against the actual upstream module. Their axiom reports contain only `propext` and `Quot.sound`.

The parity proof is not necessarily shorter as an isolated four-line script: the original enumeration is already compactly written. The improvement is an explicit reusable reason for the result, with less dependence on the particular radix exponent or digit case count. Shared-helper overhead should be included when assessing a one-off replacement.

4.2 Factor the correct cyclic residue statements

Near-identical block-residue proofs appear in CurrentClause2011, CurrentReflectedGeometry2011, NativeLocalMask54, and NativeCleanBands54. The useful common parameter is the block length L, with positions in $\mathbb{Z}/(NL)\mathbb{Z}$. The checked generic identities are

$$\mathrm{val}(b+e) \bmod L = (\mathrm{val}(b)+e) \bmod L,$$

$$(\mathrm{val}(k-e)+e) \bmod L = \mathrm{val}(k) \bmod L.$$

Both follow by casting into the full cyclic modulus, then restricting modular equality along $L \mid NL$. The supplied statements use `[NeZero (N * L)]`.

The subtraction identity is intentionally written with addition of e after taking the representative. Replacing cyclic subtraction by truncated natural subtraction would break wrapped positions. A common arithmetic module can export these facts below both the current and legacy calibration layers. CurrentCellTranslation2011 can then consume them while retaining its explicit wrapped and unwrapped source-position theorems.

Checked: the generic residue and scaled-arithmetic lemmas compile. Migrating every current and legacy caller is still a proposed follow-up. Their common arithmetic does not identify the different masks, offsets, or tile calibrations.

5 Shared scanner machinery without changing raw-input soundness

AnnotatedAxiomContext, AnnotatedBoundScan, AnnotatedLineScan, AnnotatedContext, and AnnotatedRenameScan all define a numeric scanner by primitive recursion on fuel. The transition function may have additional fixed parameters, but after those are supplied it is an endofunction on states.

The checked `ScannerCore` defines exactly the existing recursion pattern:

```
def scan (step : S -> S) (fuel : Nat) (initial : S) : S :=
  fuel.rec initial (fun _ acc => step acc)
```

It proves the successor law, preservation of a fixed state, preservation of an invariant, and preservation of "reject or good." All four axiom reports are empty. Existing scanner definitions can remain unchanged and use these facts by definitional equality; a supplied example checks the raw recursion directly.

For the soundness interface, let r be the rejecting state and suppose

$$T(r) = r, \qquad G(s) \Longrightarrow T(s) = r \ \vee \ G(T(s)).$$

Then starting from a good state, every fuel-bounded iterate is rejecting or good. This is the invariant $P(s) := (s = r \vee G(s))$ under the ordinary iteration theorem. The disjunction need not be exclusive.

For a concrete proof checker, G can say that a raw numeric state encodes an actual history of provable judgments. The transition proof must still establish that invariant from the local checker result. It must not assume that arbitrary raw inputs already parse as valid formulas or derivations. Likewise, the helper does not prove termination, enough fuel, successful acceptance, or the primitive-recursive realization of the scanner program. Those obligations remain in the concrete modules.

The first migration should target `scan_zero`, `scan_succ`, and `scan_done`; these are low-risk direct uses. Only then should the common shape of `scan_sound` be factored, with the raw-input theorem and

explicit program code checked again. A change to a program constructor requires separate attention to the existing code-size claims.

6 One finite-fiber identity for folded stencils

NativeCyclicConvolution proves `mass_foldStencil`, `convolution_foldStencil`, and the finite-sum step inside `word_foldStencil` by repeatedly swapping sums and collapsing an equality filter. All are instances of

$$\sum_{k\in K}\left(\sum_{i\in I:p(i)=k} a_i\right) w_k = \sum_{i\in I} a_i w_{p(i)}. \tag{2}$$

The supplied `FinitePushforward.sum_fiber_mul` proves this for finite index types, an arbitrary map $p : I \to K$, and a semiring. Multiplication need not be commutative, because the order $a_i w_{p(i)}$ is retained. Decidable equality on K is used to express the finite filter.

Three specializations are immediate: $w_k = 1$ for mass, $w_k = c(q-k)$ for convolution at q, and $w_k = \rho^{\mathrm{val}(k)}$ for a word. For the last specialization, the existing `pow_mod_length` argument is still required to replace the reduced exponent by the original exponent modulo $\rho^n - 1$.

Checked: the generic identity compiles. It assumes no injectivity of p, so all collisions are retained. It does not establish no-carry bounds, mask compatibility, or uniqueness of a word representation. A safe integration replaces the repeated finite-sum rearrangement and preserves those separate arithmetic obligations.

This is a useful example of a small abstraction with several interpretations. It makes the collision behavior obvious at the interface instead of requiring each proof reader to reconstruct it from a double sum.

7 Generated circuits: reduce proof scaffolding with existing checkers

7.1 Replace repetitive node interfaces selectively

Module	Physical lines	Node equations	Power equations
`GapCircuitSteps66`	3,122	484	86
`SplitCircuitSteps59`	3,445	515	107
Total	6,567	999	193

Both files explicitly identify themselves as generated. Their local node equations already apply `ArithmeticDAG.run_node_equation` from ArithmeticDAGSemantics. Thus the mathematical invariant is already shared; the remaining duplication is the interface and its generated normalization.

Proposed: expose one theorem indexed by `Fin naturalNodes.length`, giving the local equation for that node. At a use site, instantiate it and reduce only the selected node lookup. Alternatively, a small proof-producing elaborator can perform those two steps. The generator can retain named theorems at semantic boundaries such as the base, a source quantity, or an output residual, while avoiding an independent declaration for every routine node.

There is a concrete tradeoff. Existing named node equations cache their reduced types. Replacing them by repeated lookup reduction against a large literal list can make elaboration slower even if the source is

shorter. A measured migration should compare the downstream recipe modules, not just the step-file line count. Compatibility aliases and newly emitted data must be counted in the final size.

7.2 The power-label checker is already available

ArithmeticDAGPowers contains `PowerStep`, `run_power`, and `value_eq_of_powerRewrite`. These establish soundness of a table of exponent labels for the actual topological circuit. The generation scripts already compute labels of this form: the base receives label one, and a multiplication of two labelled predecessors receives the sum of their labels.

Proposed: emit the label table as data, prove its finite local obligations by kernel-checked computation, and apply `run_power` once. Named powers can then be short specializations. This is preferable to inventing a second symbolic-power correctness mechanism or repeatedly invoking `ring` after each predecessor rewrite.

The 193 generated power equations identify the candidate population; they are not a promised 193-theorem net saving. The label data and API aliases have a cost. The proof must retain the actual circuit, its topological order, the correct base, operand labels, and index bounds. This migration has not been implemented in the supplied patches.

8 Expression and compiler architecture

8.1 Centralize residual-square semantics and degree bounds

DAGQuartic, NaturalDAGQuartic, and QuadraticRegisterSystem repeat an evaluator lemma for `QuarticSyntax.sumExpressions` and a theorem that a finite sum of squared integer residuals vanishes exactly when every residual vanishes. Their proofs already use the library's `Finset.sum_mul_self_eq_zero_iff`.

Proposed: put list evaluation and the finite residual-square interface below the three compiler layers. Each compiler then supplies only its own residual family. A companion degree theorem should derive a quartic upper bound from quadratic bounds on the residuals. This avoids reproving both the semantic and polynomial-size bookkeeping for every witness-layout variant.

The zero-set theorem needs its mathematical domain. Over $\mathbb{Z}$ the nonnegative squares cannot cancel. Over an arbitrary commutative ring the statement can fail, so a ring-generic evaluator theorem is not by itself a ring-generic residual-zero theorem. Use the existing ordered/integer interface or state exactly the additional assumptions that a broader theorem needs.

8.2 Move basic syntax facts below the consumers

`QuarticCompiler.Gates.polynomial_rename` and `NaturalRegisterRelation.rename_toPolynomial` have the same statement:

$$(\text{rename}(f, e)).\,\text{toPolynomial} = \text{MvPolynomial}.\,\text{rename}(f, e.\,\text{toPolynomial}).$$

Both are structural induction proofs. A canonical theorem in the `Expr` layer removes duplicate ownership; aliases can retain existing call sites.

Similarly, `QuadraticRegisterSystem.syntaxDegree` and `polynomial_degree_le` concern expression syntax, not register execution. Moving them into a low-level expression module allows all residual

compilers to use one syntactic degree bound. Such a bound is an upper bound; it must not replace a semantic identity or an exact degree claim that depends on cancellation.

8.3 Share the gate soundness and completeness inductions

`QuarticCompiler` contains natural/integer gate soundness and completeness proofs. `GatesSemiring` contains parallel generic semiring proofs for the same wire structure, root, and residual syntax. The existing import direction is important: `GatesSemiring` imports `QuarticCompiler`.

Proposedmigration:

1. Extract `Wire`, `root`, `count`, and `residualSyntax` into a dependency-light `GatesSyntax` module.
2. Put generic semiring evaluation, soundness, and completeness above that syntax.
3. Derive the natural/integer versions using existing `NatExpr.evalSemiring_nat` and `Expr.eval_eq_zero_iff_splitNatural`.
4. Retain the executable constructors and verify exact witness and operation counts.

Simply calling a generic gate theorem from the current compiler without first extracting syntax would create an import cycle. The real simplification is one owner for each induction and short transport results at the boundaries.

8.4 Respect the object-language boundary

The `Object*` modules prove arithmetic facts inside the repository's sealed ZFC calculus. `ObjectNaturalSemiring` ranges over the model's entire internal natural numbers, including any nonstandard elements. `ObjectResidualSquares` uses this interpretation to obtain an internal residual-conjunction statement.

A proof about ordinary Lean naturals alone cannot replace such a theorem. The useful refactor is to identify a generic algebraic statement that holds for the already constructed internal semiring, then use the explicit semantic and syntactic transport theorems. Avoid replacing internal assignments by standard numerals or treating an external semantic proof as an internal derivation. This boundary is central to preserving the intended consistency claim.

9 Further targeted opportunities

9.1 Bundle carry witnesses with named fields

`NativeWindowCarry.source_convolution_carries` and `CurrentWindowCarry2011.source_carries` return carry and digit functions with several nested facts: carry bounds, digit bounds, quotient recurrence, the carrying equation, the reconstructed word, and mask compatibility. Repeatedly unpacking these long conjunctions increases argument-management work.

Proposed: define a small record for the generic carry solution, with named fields for the recurrence and bounded carries, then put source-specific word and mask facts in a separate wrapper if needed. Where a digit is definitionally the remainder, its bound and carrying equation should be derived once from division with a positive radix. A record improves theorem navigation and makes it harder to supply a fact about the wrong digit array.

The record must not collapse local carry reset into a global no-carry claim. Nor should it introduce a hidden temporal-alignment premise. Those distinctions are present in the current arithmetic and remain essential. This proposal is about a clearer interface; no line reduction is measured for it yet.

9.2 Use standard membership lemmas for small inductions

The private `AnnotatedProofChecker.mem_of_lookup` repeats a list/index induction to infer $j \in \text{seen}$ from `seen[i]? = some j`. The pinned core library already provides `List.mem_of_getElem?`. The supplied `ListLookup` file checks the corresponding generic theorem directly, with only `propext` in its axiom report.

This is a checked API-level cleanup, not a replayed patch to the entire annotated checker. Its surrounding natural-deduction rule proof has genuinely distinct cases. Explicit rule constructors can remain visible while standard list facts discharge the repeated premise-membership obligations.

9.3 Use existing structural proofs as patterns

Two recent modules already illustrate the intended direction. `WangTiles942Permutation` proves a structural left inverse for the actual tile table, then proves injectivity and rank properties. It does not perform a quadratic comparison of every tile pair. `NativeFieldCycles2011` uses periodicity and modular arithmetic to exclude the remaining offsets, including a short two-step residue obstruction where no finiteness is needed.

These are existing achievements, not new results of this audit. They suggest a method for future proofs: identify the structural inverse, invariant, or local residue obstruction before generating a large finite case analysis. In `CurrentKernel2011`, small batches of kernel-computed finite sums also serve as deliberate elaboration boundaries. Removing those boundaries to obtain one large decision call needs performance evidence.

10 Implementation order and validation policy

Stage	Change	Evidence in this package	Next integration step
1	Assignment naturality	Complete checked patch	Apply in a branch; replay direct consumers
1	Coefficient envelope schema	Complete checked patch	Check downstream coefficient-bound consumers
1	Residue and scaled arithmetic	Checked generic helpers	Migrate callers with original signatures
1	List membership alias	Checked library API	Replay annotated checker
2	Scanner fixed-point/iteration facts	Checked generic helper	Replace basic lemmas before soundness invariants

Stage	Change	Evidence in this package	Next integration step
2	Weighted finite fibers	Checked generic helper	Specialize mass, convolution, word sum step
2	Expression renaming and residual-square API	Source-grounded proposal	Move low-level lemmas; retain aliases
3	Gate semantics shared over semirings	Architecture proposal	Extract syntax first; avoid import cycle
3	Indexed circuit steps and power labels	Existing checker identified	Pilot one recipe; measure elaboration and memory
3	Carry record and generic block geometry	Interface proposal	Keep explicit source and phase obligations

For a proof-only patch, review the theorem statement and executable data first. Then compile the complete changed module against the pinned dependencies, inspect its axiom report, and replay the direct consumers that depend on its API or reduction behavior. For a structural change, add an exact equality or transport theorem when data preservation is part of the claim. Source-level shortening by itself is not evidence that a witness representation is unchanged.

The included experiments check complete patched modules independently, so a candidate is not accidentally accepted by importing the old theorem with the same name. Additive prototypes can import old modules for definitions, but their proofs must not invoke the old theorem they purport to replace. The rotation alternatives meet this condition.

10.1 Trusted computation and simplification boundaries

The existing development uses kernel-checked finite computation. The suggested certificate approach retains that property. Lean's documentation distinguishes `decide +kernel` from `native_decide`/`decide +native`, which uses native evaluation and changes the trusted dependencies of a proof [4, 5]. This report does not propose changing that trust boundary to save source lines.

Use explicit semantic rewrite lemmas around representations, scopes, and assignments. Lean's `simp only` permits a deliberately bounded rewrite set [6]; it is useful at those interfaces. Broad simplification is often appropriate for small constructor goals, but expanding an entire circuit or coefficient recipe as a global simp rule can obscure the intended boundaries.

11 Validation results and reproducibility

The bundle contains source hashes, patch checks, compilation logs, and machine-readable records of successful runs. The generic residue, scaled arithmetic, finite-fiber, scanner, assignment, and list-lookup interfaces are compiled with the pinned toolchain. The detailed status of each final source is recorded in `validation/final-validation.json`.

The reported axiom sets for the new algebraic and representation proofs are subsets of {`propext`, `Classical.choice`, `Quot.sound`}. The four scanner lemmas require no axioms. No new `sorry`, custom axiom, or native-evaluation oracle is used. A separate source review checked the packaged generic interfaces for hidden hypothesis strengthening and circularity; that review is included as an evidence

note and is not described as a second compilation run.

The complete DiophantineConsistency library and all downstream consumers were not rebuilt. The generic scanner and finite-sum abstractions have not been installed throughout the project. Architectural proposals remain proposals. These limits are reflected in the status labels rather than being hidden by the existence of a successful small example.

Individual compilation times are retained for reproducibility, not interpreted as performance benchmarks. For example, one representation replay took 3.90 s for the original and 5.01 s for the patched file under concurrent load. Those observations do not demonstrate a speedup. Other checks ran under substantial shared CPU and disk pressure. A serious performance comparison should isolate the same complete target, use the same cached imports and output mode, repeat runs, and report peak memory as well as elapsed time.

11.1 How to use the bundle

Start from the pinned ProveIt checkout with its Lean dependencies available. The `README.md` describes the exact commands. Run `scripts/verify.py` against that checkout to check patch applicability, compile the prototypes and full patched copies, and record a fresh validation result. The script leaves the checkout's source files unchanged. It expects the listed baseline dependency modules to have already been built; this avoids silently launching a full project build.

The original source remains authoritative. The two unified patches are intended for review on an integration branch. The `patched/` copies make the complete proposed modules easy to inspect. The full local toolchain, package caches, and compiled objects are deliberately not included in the archive; the pinned versions and commands are sufficient to recreate them.

12 Further research questions

1. Can one small certified arithmetic-recipe interface cover the repeated coefficient families while retaining their distinct sharp bounds? The useful abstraction would expose a finite set of coefficient expressions and an explicit sign basis, rather than treating the final inequality as an opaque tactic call.
2. What is the best granularity of cached circuit equations? An indexed theorem minimizes declarations, while named intermediate equations cache normalization. A hybrid selected by actual downstream use may be better than either extreme.
3. Can the existing exponent-label checker be extended to additional arithmetic invariants, such as syntactic degree or coefficient support, with one shared local soundness theorem for circuit folds?
4. How much of the annotated checker can be expressed through one scanner invariant without obscuring arbitrary-raw-input soundness or changing the primitive-recursive code whose size is counted?
5. Which internal-natural algebra results can be transported through the established semiring interpretation, and which depend essentially on the object-language encoding and sealing machinery? A precise inventory would prevent both duplicate inductions and invalid external-to-internal shortcuts.
6. Can a parameterized cyclic-block library serve both the legacy and current arithmetic without making calibration-dependent assumptions implicit? The shared residue results are a small verified first step; masks and field positions should remain explicit data with proved interfaces.

References

[1] V. Reshetnikov et al., *ProveIt, DiophantineConsistency*, pinned commit 60e9c5d620c6c38. . . . Module links throughout this report refer to this snapshot.

[2] ProveIt, lean-toolchain and lake-manifest.json.

[3] The mathlib community, mathlib at 81a5d257c8e410db. . . .

[4] Lean Language Reference, Tactic Reference. Retrieved 26 October 2026. Version-specific claims in the experiments are additionally checked by the pinned compiler.

[5] Lean Language Reference, Validating a Lean Proof. Retrieved 26 October 2026.

[6] Lean Language Reference, Invoking the Simplifier. Retrieved 26 October 2026.